02-3、接口自动化Agent开发实战

1、接口文档数据清洗

1、openapi文档清洗方案

```
# @Author  : 木森
# @weixin: python771

import json
from typing import Dict, Any, List, Union


class OpenAPIAggregator:
    """
    OpenAPI 聚合器，用于加载、解析和聚合 OpenAPI 规范中的接口信息。

    参数:
    - openapi_path: OpenAPI 规范文件的路径。
    """

    def __init__(self, openapi_path: str):
        self.openapi_path = openapi_path
        self.openapi = self._load_openapi()
        self.components = self.openapi.get("components", {}).get("schemas", {})
        self.parameter_defs = self.openapi.get("components",
{}).get("parameters", {})
        self.request_body_defs = self.openapi.get("components",
{}).get("requestBodies", {})

    def _load_openapi(self) -> Dict[str, Any]:
        """
        加载 OpenAPI 规范文件。

        返回:
        - 解析后的 OpenAPI 规范字典。
        """
        with open(self.openapi_path, "r", encoding="utf-8") as f:
            return json.load(f)

    def _resolve_ref(self, ref: str) -> Any:
        """
        解析 $ref 引用。
        参数:
        - ref: 引用字符串，以 #/components 开头。

        返回:
        - 引用所指向的 schema、parameter 或 requestBody。
        """
        if ref.startswith("#/components/schemas/"):
            schema_name = ref.split("/")[-1]
            return self.components.get(schema_name, {"missing_schema":
schema_name})
```

```
        elif ref.startswith("#/components/parameters/"):
            param_name = ref.split("/")[-1]
            return self.parameter_defs.get(param_name, {"missing_param":
param_name})
        elif ref.startswith("#/components/requestBodies/"):
            body_name = ref.split("/")[-1]
            return self.request_body_defs.get(body_name,
{"missing_request_body": body_name})
        else:
            return {"unresolved_ref": ref}

    def _parse_schema(self, schema: Dict[str, Any]) -> Union[Dict[str, Any],
List[Any]]:
        """
        递归解析 schema。

        参数:
        - schema: 待解析的 schema 字典。

        返回:
        - 解析后的 schema。
        """
        if '$ref' in schema:
            schema = self._resolve_ref(schema['$ref'])

        if schema.get('type') == 'object':
            result = {}
            for prop, prop_info in schema.get('properties', {}).items():
                result[prop] = self._parse_schema(prop_info) if '$ref' in
prop_info else {
                    "type": prop_info.get("type"),
                    "description": prop_info.get("description", "")
                }
            return result
        elif schema.get('type') == 'array' and 'items' in schema:
            return [self._parse_schema(schema['items'])]
        else:
            return {
                "type": schema.get("type"),
                "description": schema.get("description", "")
            }

    def aggregate(self) -> List[Dict[str, Any]]:
        """
        聚合 OpenAPI 规范中的接口信息。

        返回:
        - 包含所有接口信息的列表。
        """
        paths = self.openapi.get("paths", {})
        api_list = []
        for path, methods in paths.items():
            for method, details in methods.items():
                api_entry = {
                    "path": path,
                    "method": method.upper(),
                    "summary": details.get("summary", ""),
                    "tags": details.get("tags", []),
```

```
                    "description": details.get("description", ""),
                    "parameters": [],
                    "requestBody": {},
                    "responses": {}
                }

                # parameters
                for param in details.get("parameters", []):
                    if "$ref" in param:
                        param = self._resolve_ref(param["$ref"])
                    schema = param.get("schema", {})
                    api_entry["parameters"].append({
                        "name": param.get("name"),
                        "in": param.get("in"),
                        "required": param.get("required", False),
                        "type": schema.get("type"),
                        "description": param.get("description", "")
                    })

                # request body
                req_body = details.get("requestBody", {})
                if "$ref" in req_body:
                    req_body = self._resolve_ref(req_body["$ref"])
                content = req_body.get("content", {})
                if "application/json" in content:
                    schema = content["application/json"].get("schema", {})
                    api_entry["requestBody"] = self._parse_schema(schema)

                # responses
                for status_code, resp in details.get("responses", {}).items():
                    content = resp.get("content", {})
                    if "application/json" in content:
                        schema = content["application/json"].get("schema", {})
                        api_entry["responses"][status_code] =
self._parse_schema(schema)

                api_list.append(api_entry)
        return api_list

    def save_to_json(self, output_path: str, data: List[Dict[str, Any]]):
        """
        将聚合后的接口信息保存为 JSON 文件。
        参数:
        - output_path: 输出文件的路径。
        - data: 聚合后的接口信息列表。
        """
        with open(output_path, "w", encoding="utf-8") as f:
            json.dump(data, f, ensure_ascii=False, indent=2)
        print(f"已保存聚合后的接口信息，共 {len(data)} 条接口 → {output_path}")


# 使用示例
if __name__ == "__main__":
    aggregator = OpenAPIAggregator("../../data/openapi.json")
    api_info = aggregator.aggregate()
    for api in api_info:
        print(api)
```

2、其他的接口文档

- 可以导出结构化数据的文档，那么就可以自己写代码进行处理成统一的格式
- 如果是非结构化的数据文档，可以使用大模型进行清洗(参考第一阶段需求文档的数据清洗方案)

- 接口文档--->迭代升级会发生变动---->有些接口的文档需要更新(维护和更新)
- document存储向量库之后，会返回uuid(文档数据的唯一标识)
  - 更新向量数据库中某个接口的文档(某个需求的文档)，得使用uuid去更新
- mysql、sqlite(文档的唯一标识uuid，基本的字段信息）+接口文档的详细数据向量数据库中

2、API数据存储方案设计

◇ 方案一：向量化存储（如 Weaviate / FAISS)

- **优点**：
  - 支持**语义检索**，可以通过自然语言模糊匹配相关接口，更适合结合大模型问答系统
  - 易于扩展多模态（如描述性文本、错误信息等）的检索需求。
- **缺点**：
  - 不支持复杂结构化查询（如分页、分类、筛选等前端需求）。
  - 接口变更后需要**重新生成嵌入并同步更新向量库**，维护成本高。
  - 不适合完整保存接口结构信息（如参数、响应字段表格等）。

◇ 方案二：关系型数据库存储（如 MySQL ）

- **优点**：
  - 支持结构化存储，适合**前端接口列表展示、分页、分类、精确搜索与跳转**等功能。
  - 修改、更新、批量操作接口数据更方便，在生成时也可以直接引用接口。
- **缺点**：
  - **不具备语义检索能力**，无法处理用户模糊提问（如“怎么注册用户”）。
  - 需要人工构建复杂的查询条件或关键词索引来模拟“智能问答”场景。

◇ 方案三：向量 + 数据库双存储（推荐）

- ☑ **优点**：
  - 结合两者优点：语义检索通过向量库完成，结构化管理和展示通过数据库实现。
  - 更利于**接口迭代升级的可维护性和后续的可视化管理**。

3、接口数据存储格式设计

```
{
    "project":"所属项目名称",
    "module":"所属模块名称",
    "tag": "接口标签",
    "interface_id":"接口的唯一ID，存储向量数据库的uuid",
    "method":"请求方法",
```

```
    "path":"接口路径",
    "summary":"接口名称",
    "description":"接口写详细的业务描述",
    "parameters":"查询参数说明",
    "requestBody":"请求体的说明",,
    "responses":"响应体的说明",
    "updated_at":"更新时间",
    "version":"版本号"
}
```

1、注意点：唯一接口ID的生成

使用如下三要素拼接并哈希：

- `method`（如：GET）
- `path`（如：/api/users）
- 当前时间戳（精确到毫秒）

然后用 uuid生成唯一 ID，确保不同时间生成的接口即便其他字段一致也不会重复。

```python
import uuid
import time

def generate_interface_id(method: str, path: str) -> str:
    """生成接口的唯一ID"""
    timestamp = str(int(time.time() * 1000))  # 毫秒时间戳
    raw_string = f"{method.upper()}::{path}::{timestamp}"
    return uuid.uuid3(raw_string)
```

4、生产级向量库Weaviate

1、Weaviate 向量数据库简介

- Weaviate 是什么？（开源、支持外部嵌入、多种客户端）
- 核心特点：
  - 支持 schema 定义
  - 支持 RESTful API / gRPC 双协议
  - 支持外部嵌入向量（如 OpenAI、bge-m3 等）
- 官网地址：https://weaviate.io/developers/weaviate/installation/docker-compose
- 使用文档：https://weaviate.io/developers/weaviate/client-libraries/python#generate
- langchian集成文档：https://python.langchain.com/docs/integrations/vectorstores/weaviate/#persistence

2、Weaviate 安装与部署

用mac的同学库安装docker后，可以直接在本地运行部署，

用window的同学可以先

- 方案一：安装docker-desktop,
- 方案二： 或者直接买个几十块一年的云服务器
- 方案三：或者在window安装wsl, linux环境再安装docker

> 1. **windows以管理员身份打开 PowerShell**：
> - 按 Win + X，选择 "Windows Terminal (管理员)" 或 "PowerShell (管理员)"
> 2. **执行以下命令安装 WSL**：
> powershell
>  ```
>  wsl --install
>  ```
> 此命令会自动完成以下操作：
> - 启用 WSL 功能
> - 启用虚拟机平台功能
> - 下载并安装默认的 Ubuntu 发行版

- （Docker部署安装：

  ```
  docker run -p 8080:8080 -p 50051:50051  semitechnologies/weaviate:1.30.3
  ```

- 配置项说明（端口、安全模式、持久化路径等）
- 浏览器访问 Weaviate UI：http://localhost:8080

安装第三方库：

```
pip install  weaviate-client
pip install langchain-weaviate
```

3、创建 Weaviate 客户端

```
import weaviate

# 自定义连接地址
client = weaviate.connect_to_custom(
    http_host="192.168.0.108",
    http_port=8080,
    grpc_host="192.168.0.108",
    grpc_port=50051,
    http_secure=False,
    grpc_secure=False
)
# 连接本地的向量库
```

4、创建 集合

数据在存储之前需求创建一个存储空间

```
# 创建一个数据集
connection = client.collections.create_from_dict({
    # 类：相当于关系型数据库中的"表"，
    "class": "ApiInterface01",
    # 定义属性列表(在向量化之外要存储的数据字段放在这里)
    "properties": [
        {"name": "text", "dataType": ["text"]}
    ],
    # 设置为none 使用自定义的向量模型
    "vectorizer": "none"
})
```

5、插入数据

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    model='Pro/BAAI/bge-m3',
    api_key=os.getenv('API_KEY'),
    base_url="https://api.siliconflow.cn/v1"
)
# 文本向量化处理
vector = embeddings.embed_query("你好")
# 插入到weaviate中
collections.data.insert(
    properties={"text": "你好"},
    vector=vector
)
```

6、查询数据

查询结果中的distance表示数据的向量计算距离，值越小，表示越接近。

```
# 内容向量化处理
vector = self.embeddings.embed_query(text)
# 进行向量检索
results = self.collection.query.near_vector(
    near_vector=vector,
    limit=top_k,
    return_metadata=MetadataQuery(distance=True),
)
# 进行关键词检索
results = self.collection.query.bm25(
    query=query,
    limit=top_k,
    # 是否返回元数据
    return_metadata=MetadataQuery(distance=True),
)

# 混合检索：向量 + 关键词
from weaviate.collections.classes.grpc import HybridFusion
vector = self.embeddings.embed_query(query)

results = self.collection.query.hybrid(
    query=query,
    vector=vector,
```

```
    limit=top_k,
    alpha=alpha,
    fusion_type=HybridFusion.RELATIVE_SCORE,
    return_metadata=MetadataQuery(distance=True),
)
```

7、常用操作的封装

```
# @Author  : 木森
# @weixin: python771
import os
import uuid
import dotenv
import jieba
import weaviate
from weaviate.collections import Collection
from weaviate.classes.query import MetadataQuery
from langchain_openai import OpenAIEmbeddings
from weaviate.collections.classes.grpc import HybridFusion

dotenv.load_dotenv("G:/AI/上课代码/LangchainProjectDemo/DeepSeek/.env")

class WeaviateVectorManager:

    def __init__(self, host: str, port: int, class_name: str = "Document"):
        self.class_name = class_name
        self.client = weaviate.connect_to_custom(
            http_host=host,
            http_port=port,
            http_secure=False,
            grpc_host=host,
            grpc_port=50051,
            grpc_secure=False,
        )
        self.embeddings = OpenAIEmbeddings(
            model='Pro/BAAI/bge-m3',
            api_key=os.getenv('API_KEY'),
            base_url="https://api.siliconflow.cn/v1"
        )
        self._init_collection()

    def _init_collection(self):
        """初始化集合"""
        if not self.client.collections.exists(self.class_name):
            self.client.collections.create_from_dict({
                "class": self.class_name,
                "properties": [
                    {"name": "text", "dataType": ["text"]},
                ],
                "vectorizer": "none"
            })
        self.collection: Collection =
self.client.collections.get(self.class_name)

    def add_text(self, text: str, uid) -> str:
```

```python
        """添加文本内容"""
        vector = self.embeddings.embed_query(text)
        self.collection.data.insert(
            uuid=uid,
            properties={"text": text},
            vector=vector
        )
        return uid

    def delete_by_uuid(self, uid: str) -> bool:
        """根据id删除内容"""
        return self.collection.data.delete_by_uuid(uuid=uid)

    def update_text(self, uid: str, new_text: str) -> bool:
        """更新内容"""
        vector = self.embeddings.embed_query(new_text)
        return self.collection.data.update(
            uuid=uid,
            properties={"text": new_text},
            vector=vector
        )

    def search(self, text: str, top_k: int = 5):
        """内容检索"""
        # 内容向量化处理
        vector = self.embeddings.embed_query(text)
        # 进行向量检索
        results = self.collection.query.near_vector(
            near_vector=vector,
            limit=top_k,
            return_metadata=MetadataQuery(distance=True),
        )
        return results

    def hybrid_search(self, query: str, top_k: int = 5, alpha: float = 0.6):
        """
        混合检索：向量 + 关键词
        :param query: 查询文本
        :param top_k: 返回数量
        :param alpha: 融合参数（0 = 只关键词，1 = 只向量）
        """
        from weaviate.collections.classes.grpc import HybridFusion
        vector = self.embeddings.embed_query(query)
        results = self.collection.query.hybrid(
            query=query,
            vector=vector,
            limit=top_k,
            alpha=alpha,
            fusion_type=HybridFusion.RELATIVE_SCORE,
            return_metadata=MetadataQuery(distance=True),
        )
        return results

    def close(self):
        self.client.close()


if __name__ == '__main__':
```

```
    manager = WeaviateVectorManager(host="192.168.0.108", port=8080,
class_name="demo2")
    # uid = manager.add_text("今天天气不错")
    res = manager.hybrid_search("天气不错")
    print(res)
    manager.close()
```

5、关系型数据库Mysql存储

字段名	类型	允许空	键	描述
id	INT	NO	PRI	自增主键
interface_id	VARCHAR(100)	NO	UNI	接口唯一标识符uuid
project	VARCHAR(100)	YES		所属项目名称
module	VARCHAR(100)	YES		所属模块名称
tag	VARCHAR(100)	YES		接口标签
method	VARCHAR(10)	No		HTTP 请求方法（GET/POST等）
path	VARCHAR(255)	No		接口路径
summary	TEXT	YES		接口摘要
description	TEXT	YES		接口详细描述
parameters	JSON	YES		查询参数说明（JSON格式）
requestBody	JSON	YES		请求体结构（JSON格式）
responses	JSON	YES		响应体结构（JSON格式）
version	VARCHAR(20)	YES		接口版本号
updated_at	DATETIME	YES		最后更新时间

1、sql建表语句

```
CREATE TABLE api_interface (
    id INT AUTO_INCREMENT PRIMARY KEY COMMENT '自增主键ID',
    interface_id VARCHAR(100) NOT NULL UNIQUE COMMENT '接口全局唯一标识符，建议使用
UUID或项目前缀+模块+名称的哈希值',
    project VARCHAR(100) COMMENT '所属项目/系统名称，如"用户中心"、"订单系统"',
    module VARCHAR(100) COMMENT '所属模块名称，如"用户认证"、"支付管理"',
    tag VARCHAR(100) COMMENT '接口分类标签，多个标签可用逗号分隔，如"auth,login"',
    method VARCHAR(10) NOT NULL COMMENT 'HTTP请求方法：GET/POST/PUT/DELETE/PATCH
等',
    path VARCHAR(255) NOT NULL COMMENT '接口路径，不包含基地址，如"/user/login"',
    summary TEXT COMMENT '接口简要说明，50字以内概括',
    description TEXT COMMENT '接口详细描述，包含业务逻辑和使用场景说明',
    parameters JSON COMMENT '查询参数说明，JSON数组格式，包含
name/type/required/description等字段',
```

```
    requestBody JSON COMMENT '请求体结构，包含JSON Schema格式定义和示例',
    responses JSON COMMENT '响应体结构，包含不同状态码的响应示例和说明',
    version VARCHAR(20) COMMENT '接口版本号，建议使用语义化版本如1.0.2',
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
COMMENT '最后更新时间，自动维护'
)
```

2、添加接口

```
def add_interface(data: dict):
    """新增接口数据，若存在相同 interface_id 则报错"""
    conn = pymysql.connect(**db_config)
    cursor = conn.cursor()

    # 判断是否已存在
    cursor.execute("SELECT 1 FROM api_interface WHERE interface_id = %s",
(data["interface_id"],))
    if cursor.fetchone():
        cursor.close()
        conn.close()
        raise ValueError(f"接口 ID 已存在：{data['interface_id']}")

    # 插入数据库
    sql = """
    INSERT INTO api_interface (
        interface_id, project, module, tag, method, path, summary,
        description, parameters, requestBody, responses, updated_at, version
    )
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    """
    cursor.execute(sql, (
        data["interface_id"],
        data.get("project"),
        data.get("module"),
        data.get("tag"),
        data["method"],
        data["path"],
        data.get("summary"),
        data.get("description"),
        data.get("parameters"),
        data.get("requestBody"),
        data.get("responses"),
        data.get("updated_at"),
        data.get("version"),
    ))
    conn.commit()
    cursor.close()
    conn.close()
```

3、更新接口

```
def update_interface(data: dict):
    """更新已存在接口的数据"""
    conn = pymysql.connect(**db_config)
```

```
    cursor = conn.cursor()

    # 判断是否存在
    cursor.execute("SELECT 1 FROM api_interface WHERE interface_id = %s",
(data["interface_id"],))
    if not cursor.fetchone():
        cursor.close()
        conn.close()
        raise ValueError(f"接口 ID 不存在: {data['interface_id']}")

    # 执行更新
    sql = """
    UPDATE api_interface SET
        project = %s,
        module = %s,
        tag = %s,
        method = %s,
        path = %s,
        summary = %s,
        description = %s,
        parameters = %s,
        requestBody = %s,
        responses = %s,
        updated_at = %s,
        version = %s
    WHERE interface_id = %s
    """
    cursor.execute(sql, (
        data.get("project"),
        data.get("module"),
        data.get("tag"),
        data["method"],
        data["path"],
        data.get("summary"),
        data.get("description"),
        data.get("parameters"),
        data.get("requestBody"),
        data.get("responses"),
        data.get("updated_at"),
        data.get("version"),
        data["interface_id"]
    ))
    conn.commit()
    cursor.close()
    conn.close()
```

4、删除接口

```python
def delete_interface(interface_id: str):
    conn = pymysql.connect(**db_config)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM api_interface WHERE interface_id=%s",
(interface_id,))
    conn.commit()
    cursor.close()
    conn.close()
```

6、向量+数据库双存储